GML_06_02-feature-Selection.R

frank

2024-05-06

```
# Author Frank Lehrbass, May 2022
# Ein R Skript für das FOM Zertifikat
# nur zum Zwecke der Lehre
# KEINE GARANTIE bzgl Korrektheit o.a.
# This R coding is WITHOUT ANY WARRANTY

#Zuvor verwendete Variablen leeren
rm(list=ls(all=TRUE))

knitr::opts_chunk$set(message = FALSE)
knitr::opts_chunk$set(warning = FALSE)

# Bibliotheken importieren

library(lmtest) # Regressionsdiagnostik
```

```
## Warning: Paket 'lmtest' wurde unter R Version 4.3.3 erstellt
```

```
## Lade nötiges Paket: zoo
```

```
##
## Attache Paket: 'zoo'
```

```
## Die folgenden Objekte sind maskiert von 'package:base':
##
##     as.Date, as.Date.numeric
```

```
library(sandwich) #HAC SE
```

```
## Warning: Paket 'sandwich' wurde unter R Version 4.3.3 erstellt
```

```
library(car) #VIF und Co
```

```
## Warning: Paket 'car' wurde unter R Version 4.3.3 erstellt
```

```
## Lade nötiges Paket: carData
```

```
library(leaps) #for regsubsets
library(gamlr) #for LASSO etc
```

```
## Lade nötiges Paket: Matrix
```

```
library(tseries) #adf.test
```

```
## Registered S3 method overwritten by 'quantmod':
##   method            from
##   as.zoo.data.frame zoo
```

```
library(corrplot) #for corr visualization
```

```
## Warning: Paket 'corrplot' wurde unter R Version 4.3.3 erstellt
```

```
## corrplot 0.92 loaded
```

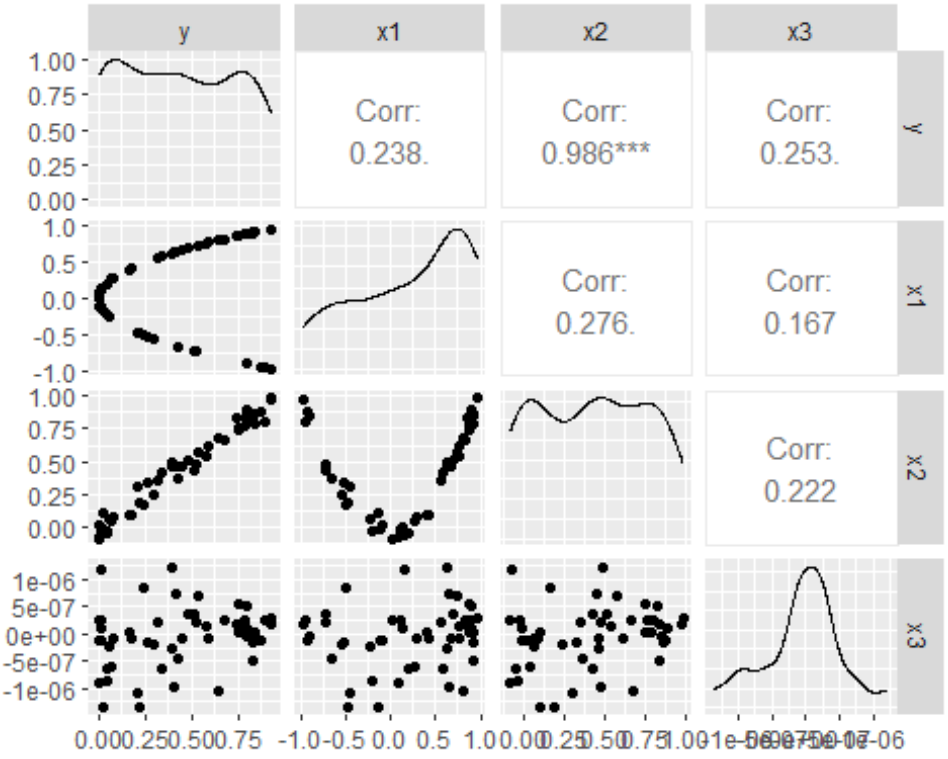
```
library(GGally)#for plotting ggpairs
```

```
## Lade nötiges Paket: ggplot2
```

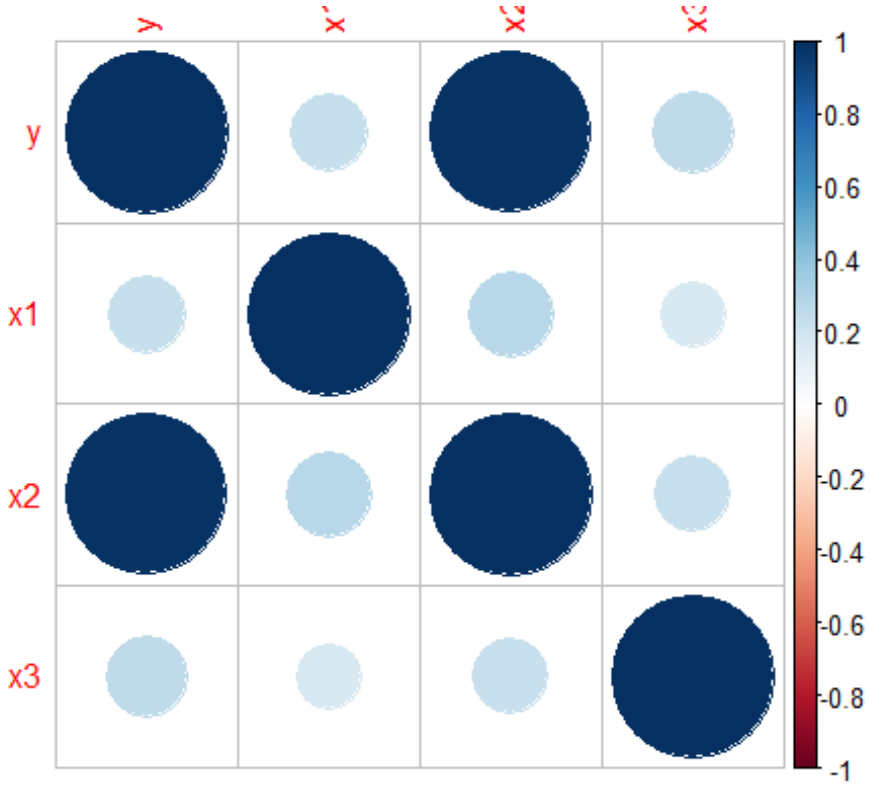
```
## Warning: Paket 'ggplot2' wurde unter R Version 4.3.3 erstellt
```

```
## Registered S3 method overwritten by 'GGally':
##   method from
##   +.gg   ggplot2
```

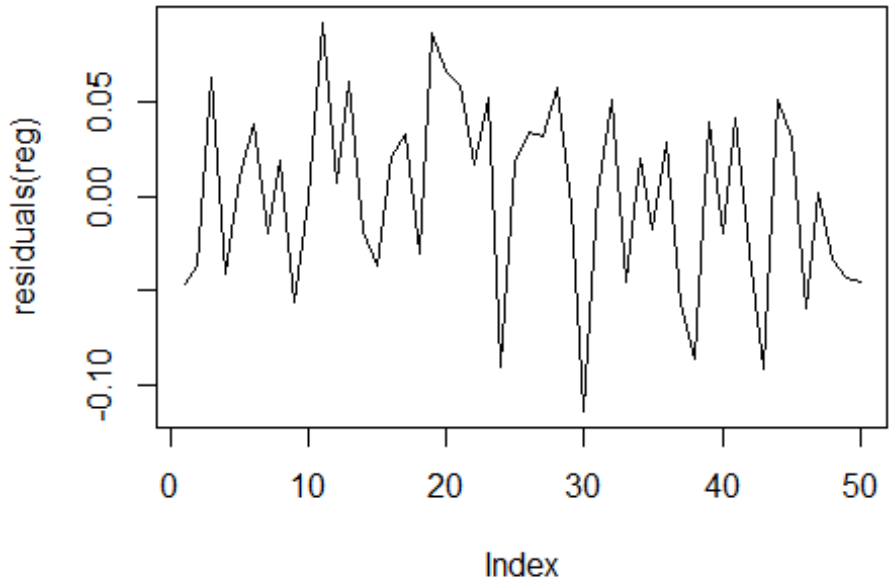
```
#for CART
library(rpart)
library(rpart.plot)

#for MLP
library(neuralnet)
library(NeuralNetTools)

# DGP

SEED = 2021

n=50 #50
n_boot = 2000 #2000 fits to big data considerations in slides
n_epochs = 100 #100 is sufficient and 1000 serves as upper limit for this toy
example
set.seed(SEED)
x1 = runif(n,-1,1)
z = runif(n,-0.1,0.1)
eps = rnorm(n,0,0.0005)
y=x1^2+eps
x2=y+z
x3=rnorm(n,0,0.0000005)

df1 = data.frame(cbind(y,x1,x2,x3))

# Visual analysis

x11()#open new window
ggpairs(df1)#first column shows: y = x1² with smallest noise!
```




```
#and y linear fct of x2 but with more noise
M=cor(df1)
x11()
corrplot(M,method="circle")
```



```r
# Linear Regression

lm1 = lm(y~., data = df1)

# Diagnostics

reg = lm1 #replace here if u want to diag another regression

## Test auf Stationaritaet: ------------------------------
adf.test(y)

##
##  Augmented Dickey-Fuller Test
##
## data:  y
## Dickey-Fuller = -3.7393, Lag order = 3, p-value = 0.03053
## alternative hypothesis: stationary

adf.test(x1)

##
##  Augmented Dickey-Fuller Test
##
## data:  x1
## Dickey-Fuller = -3.6204, Lag order = 3, p-value = 0.04036
## alternative hypothesis: stationary

#rest is stationary by construction

##Fehlspezifikation----------------------------------------
resettest(reg)#H0 No spec error

##
##  RESET test
##
## data:  reg
## RESET = 5.4236, df1 = 2, df2 = 44, p-value = 0.007844

##Hetero---------------------------------------------------
plot(residuals(reg), type='l')
```




```
#Breusch-Pagan-Test auf Heteroskedastizit?t
bptest(reg)#H0: Homoscedasticity!

##
##  studentized Breusch-Pagan test
##
## data:  reg
## BP = 0.78662, df = 3, p-value = 0.8527

##Multicoll-----------------------------------------------
vif(reg)

##       x1       x2       x3
## 1.096692 1.121438 1.065499

##conclusion----------------------------------------------
#Newey-West-Korrektur nicht noetig
summary(lm1)

##
## Call:
## lm(formula = y ~ ., data = df1)
##
## Residuals:
##       Min        1Q    Median        3Q       Max
## -0.113882 -0.037117  0.004529  0.037204  0.091722
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
```

```
## (Intercept)  3.986e-02  1.191e-02   3.346  0.00164 **
## x1          -2.130e-02  1.217e-02  -1.751  0.08669 .
## x2           9.224e-01  2.267e-02  40.684  < 2e-16 ***
## x3           2.193e+04  1.299e+04   1.688  0.09813 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.05058 on 46 degrees of freedom
## Multiple R-squared:  0.9758, Adjusted R-squared:  0.9742
## F-statistic: 617.1 on 3 and 46 DF,  p-value: < 2.2e-16

#INSIGHT-----------------------------------------------------------------
------
#choosing x1 would be the right choice if we are going to apply nlr
#However, if the learner is going to be a linear regressor, then we should
prefer x2
#important graphical inspection!!!

# Linear Regression and BOOTSTRAP technique intro

# Bootstrap regression coefficients
set.seed(SEED)
betas_boot = matrix(0,n_boot,4)
for (b in 1:n_boot){
  boot.data = df1[sample(nrow(df1), n, replace = TRUE),]
  betas_boot[b,] = coef(lm(y~., data = boot.data))
}
#ESTIMATES and their SE
cbind(matrix((apply(betas_boot, MARGIN = 2,
mean)),4,1),matrix(apply(betas_boot, MARGIN = 2, sd),4,1))

##               [,1]         [,2]
## [1,]  3.945555e-02 1.065655e-02
## [2,] -2.105952e-02 1.342983e-02
## [3,]  9.227830e-01 2.067520e-02
## [4,]  2.146193e+04 1.552692e+04

summary(lm1)

##
## Call:
## lm(formula = y ~ ., data = df1)
##
## Residuals:
##       Min        1Q    Median        3Q       Max
## -0.113882 -0.037117  0.004529  0.037204  0.091722
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  3.986e-02  1.191e-02   3.346  0.00164 **
## x1          -2.130e-02  1.217e-02  -1.751  0.08669 .
## x2           9.224e-01  2.267e-02  40.684  < 2e-16 ***
```

```
## x3           2.193e+04  1.299e+04   1.688  0.09813 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.05058 on 46 degrees of freedom
## Multiple R-squared:  0.9758, Adjusted R-squared:  0.9742
## F-statistic: 617.1 on 3 and 46 DF,  p-value: < 2.2e-16
```

```
#the residual standard error and the coefs are well matched but the SE per x
are way off

# Linear Regression and Big Data

#increase sample size, x3 gets significant
set.seed(SEED)
n * n_boot
```

```
## [1] 1e+05
```

```
big_n = n * n_boot
big_x1 = runif(big_n,-1,1)
big_z = runif(big_n,-0.1,0.1)
big_eps = rnorm(big_n,0,0.0005)
big_y=big_x1^2+big_eps
big_x2=big_y+big_z
big_x3=rnorm(big_n,0,0.0000005)

big_df1 = data.frame(cbind(big_y,big_x1,big_x2,big_x3))

big_lm1 = lm(big_y~big_x1+big_x2+big_x3)
summary(big_lm1)#WATCH! #t-value based choice is w alpha = 5%: USE x2+x3!!!
```

```
##
## Call:
## lm(formula = big_y ~ big_x1 + big_x2 + big_x3)
##
## Residuals:
##       Min        1Q    Median        3Q       Max
## -0.108929 -0.048101 -0.000066  0.047935  0.121007
##
## Coefficients:
##              Estimate Std. Error  t value Pr(>|t|)
## (Intercept)  1.187e-02  2.661e-04   44.582   <2e-16 ***
## big_x1       2.239e-04  3.100e-04    0.722   0.4702
## big_x2       9.637e-01  5.889e-04 1636.363   <2e-16 ***
## big_x3      -8.316e+02  3.595e+02   -2.313   0.0207 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.05665 on 99996 degrees of freedom
## Multiple R-squared:  0.964,  Adjusted R-squared:  0.964
## F-statistic: 8.926e+05 on 3 and 99996 DF,  p-value: < 2.2e-16
```

```r
# backward selection, start with all regressors

backward_lm1 = regsubsets(y~x1+x2+x3, data = df1, method=c("backward"))
summary(backward_lm1)

## Subset selection object
## Call: regsubsets.formula(y ~ x1 + x2 + x3, data = df1, method =
c("backward"))
## 3 Variables  (and intercept)
##    Forced in Forced out
## x1     FALSE      FALSE
## x2     FALSE      FALSE
## x3     FALSE      FALSE
## 1 subsets of each size up to 3
## Selection Algorithm: backward
##          x1  x2  x3
## 1  ( 1 ) " " "*" " "
## 2  ( 1 ) "*" "*" " "
## 3  ( 1 ) "*" "*" "*"

#it can be seen that the best 1-variables model contains only x2, as in
INSIGHT
#it can be seen that the best 2-variables model contains only x2 and x3,
again false conclusion
#HENCE: Use domain knowledge and many charts
#and look forward to MLP where functional form is specified during training

# forward selection, start with all regressors

forward_lm1 = regsubsets(y~x1+x2+x3, data = df1, method=c("forward"))
summary(forward_lm1)

## Subset selection object
## Call: regsubsets.formula(y ~ x1 + x2 + x3, data = df1, method =
c("forward"))
## 3 Variables  (and intercept)
##    Forced in Forced out
## x1     FALSE      FALSE
## x2     FALSE      FALSE
## x3     FALSE      FALSE
## 1 subsets of each size up to 3
## Selection Algorithm: forward
##          x1  x2  x3
## 1  ( 1 ) " " "*" " "
## 2  ( 1 ) "*" "*" " "
## 3  ( 1 ) "*" "*" "*"

#alternatively as in Taddy Slides
null = glm(y ~ 1, data=df1)
full = glm(y~x1+x2+x3, data = df1)
fwd = step(null, scope=formula(full), dir="forward")
```

```
## Start:  AIC=29.29
## y ~ 1
##
##        Df Deviance      AIC
## + x2    1   0.1313 -149.223
## + x3    1   4.5441   27.984
## + x1    1   4.5792   28.369
## <none>      4.8543   29.286
##
## Step:  AIC=-149.22
## y ~ x2
##
##        Df Deviance     AIC
## + x1    1  0.12499 -149.69
## + x3    1  0.12553 -149.47
## <none>     0.13129 -149.22
##
## Step:  AIC=-149.68
## y ~ x2 + x1
##
##        Df Deviance     AIC
## + x3    1  0.11769 -150.69
## <none>     0.12499 -149.69
##
## Step:  AIC=-150.69
## y ~ x2 + x1 + x3
```

```
summary(fwd)#note the presence of x3
```

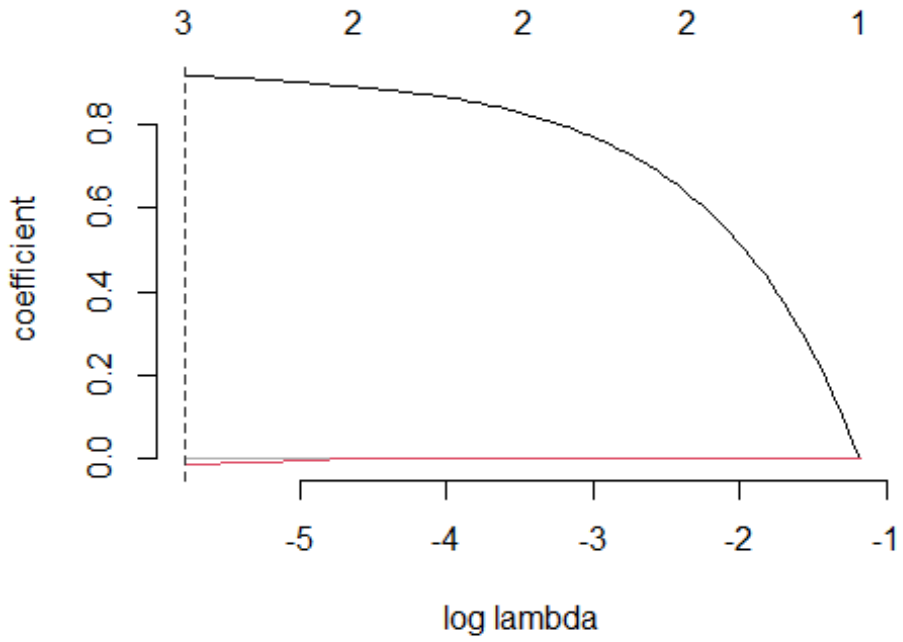
```
##
## Call:
## glm(formula = y ~ x2 + x1 + x3, data = df1)
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  3.986e-02  1.191e-02   3.346  0.00164 **
## x2           9.224e-01  2.267e-02  40.684  < 2e-16 ***
## x1          -2.130e-02  1.217e-02  -1.751  0.08669 .
## x3           2.193e+04  1.299e+04   1.688  0.09813 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for gaussian family taken to be 0.002558554)
##
##     Null deviance: 4.85426  on 49  degrees of freedom
## Residual deviance: 0.11769  on 46  degrees of freedom
## AIC: -150.69
##
## Number of Fisher Scoring iterations: 2
```

```
# LASSO selection, start with all regressors
```

```r
summary(df1)
```

```
##        y                   x1                  x2                  x3
##  Min.   :0.0004033   Min.   :-0.9666   Min.   :-0.08408   Min.   :-1.350e-
06
##  1st Qu.:0.0998827   1st Qu.:-0.2311   1st Qu.: 0.09276   1st Qu.:-2.749e-
07
##  Median :0.4134896   Median : 0.3418   Median : 0.45821   Median :-3.170e-
09
##  Mean   :0.4260907   Mean   : 0.2174   Mean   : 0.42507   Mean   :-5.516e-
08
##  3rd Qu.:0.7510345   3rd Qu.: 0.7700   3rd Qu.: 0.75174   3rd Qu.: 2.432e-
07
##  Max.   :0.9339417   Max.   : 0.9660   Max.   : 0.98504   Max.   : 1.227e-
06
```

```r
#hence we have to scale!, we do it inside function call (standardize = T)
lasso_lm1 = gamlr(cbind(x1,x2,x3), y, standardize = T) # lasso
x11()
plot(lasso_lm1)
```




```r
lasso_lm1$beta[,10] #for high lambda
```

```
##        x1        x2        x3
## 0.0000000 0.3146378 0.0000000
```

```r
lasso_lm1$beta[,50]
```

```
##        x1        x2        x3
## 0.0000000 0.8256681 0.0000000
```

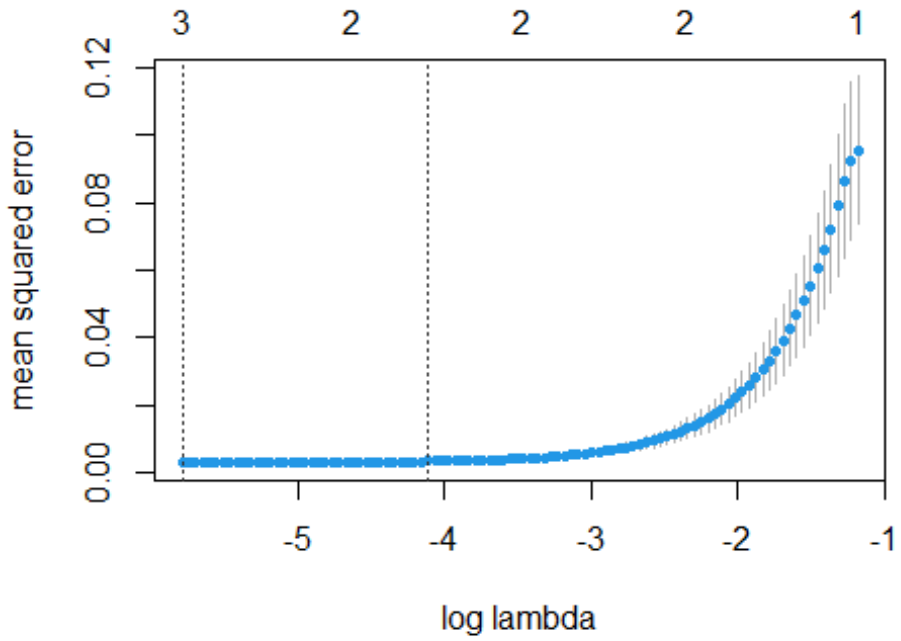
```
lasso_lm1$beta[,100] #for lowest level of lambda
```

```
##          x1          x2          x3
## -0.01208422  0.91677043  0.00000000
```

```
coef(lm1) #note that this is not the same in detail
```

```
##   (Intercept)            x1            x2            x3
##  3.985600e-02 -2.130169e-02  9.223727e-01  2.193349e+04
```

```
#(due to standardization and solving a different min-problem)
#but the main message is similar!

# LASSO selection, start with all regressors and use cross vali

lasso_lm2 = cv.gamlr(cbind(x1,x2,x3), y, standardize = T, nfold=5)
# lasso, nfold = 50 i.e.each esti uses 40 elemnts (=sample) and
#looks at error of left out set of ten elements (OOS)
x11()
plot(lasso_lm2) #see that the lowest penalty leads t lowest error out of
sample
```



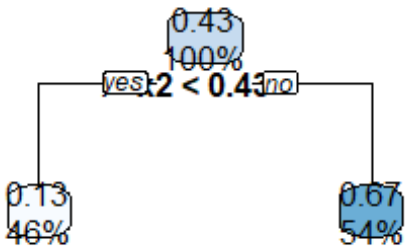

```
coef(lasso_lm2, select = "min")
```

```
## 4 x 1 sparse Matrix of class "dgCMatrix"
##                 seg100
## intercept  0.03902337
```

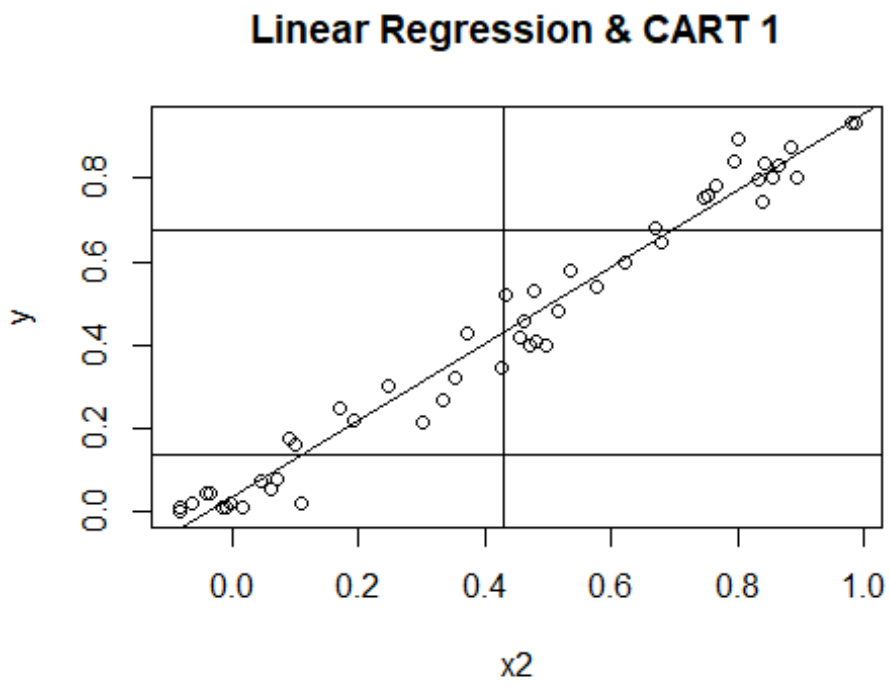
```
## x1         -0.01208422
## x2          0.91677043
## x3          .

# wrap up of econometrics

#starting point
summary(lm1)

##
## Call:
## lm(formula = y ~ ., data = df1)
##
## Residuals:
##       Min        1Q    Median        3Q       Max
## -0.113882 -0.037117  0.004529  0.037204  0.091722
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  3.986e-02  1.191e-02   3.346  0.00164 **
## x1          -2.130e-02  1.217e-02  -1.751  0.08669 .
## x2           9.224e-01  2.267e-02  40.684  < 2e-16 ***
## x3           2.193e+04  1.299e+04   1.688  0.09813 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.05058 on 46 degrees of freedom
## Multiple R-squared:  0.9758, Adjusted R-squared:  0.9742
## F-statistic: 617.1 on 3 and 46 DF,  p-value: < 2.2e-16

# LASSO et al: drop x3, Visual: use x1 squared
x1sq = x1^2
lm3 = lm(y~x1sq+x2, data = df1)
summary(lm3)

##
## Call:
## lm(formula = y ~ x1sq + x2, data = df1)
##
## Residuals:
##        Min         1Q     Median         3Q        Max
## -9.586e-04 -2.997e-04  5.668e-05  2.694e-04  9.874e-04
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept) -0.0001603  0.0001121  -1.430    0.159
## x1sq         0.9997046  0.0012434 803.984   <2e-16 ***
## x2           0.0002441  0.0011596   0.211    0.834
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.0004507 on 47 degrees of freedom
```

```
## Multiple R-squared:      1,  Adjusted R-squared:      1
## F-statistic: 1.195e+07 on 2 and 47 DF,  p-value: < 2.2e-16

# apply your first tree

#piecewise linear fct, see Schlittgen, p. 168 ff
cart1 = rpart(y~x1+x2+x3, data = df1, maxdepth = 1)
summary(cart1)

## Call:
## rpart(formula = y ~ x1 + x2 + x3, data = df1, maxdepth = 1)
##   n= 50
##
##          CP nsplit rel error    xerror       xstd
## 1 0.7490464      0 1.0000000 1.0397521 0.11546948
## 2 0.0100000      1 0.2509536 0.3445086 0.04914401
##
## Variable importance
## x2 x1 x3
## 47 35 18
##
## Node number 1: 50 observations,    complexity param=0.7490464
##   mean=0.4260907, MSE=0.09708513
##   left son=2 (23 obs) right son=3 (27 obs)
##   Primary splits:
##       x2 < 0.4278535     to the left,  improve=0.7490464, (0 missing)
##       x1 < 0.7141956     to the left,  improve=0.4374049, (0 missing)
##       x3 < -1.264893e-07 to the left,  improve=0.1500770, (0 missing)
##   Surrogate splits:
##       x1 < 0.6088468     to the left,  agree=0.88, adj=0.739, (0 split)
##       x3 < -1.264893e-07 to the left,  agree=0.72, adj=0.391, (0 split)
##
## Node number 2: 23 observations
##   mean=0.1339119, MSE=0.01691391
##
## Node number 3: 27 observations
##   mean=0.6749837, MSE=0.03071012

#https://gormanalysis.com/decision-trees-in-r-using-rpart/

#use fcts
round(cor(lm1$fitted.values,y)^2,4)

## [1] 0.9758

round(cor(rpart.predict(cart1,as.data.frame(cbind(x1,x2,x3)),rules=F),y)^2,4)
#decrease

## [1] 0.749

x11()
#compare this to CART tree result in Larose page 324, Fig. 11.4
rpart.plot(cart1, main = "CART 1")
```

CART 1

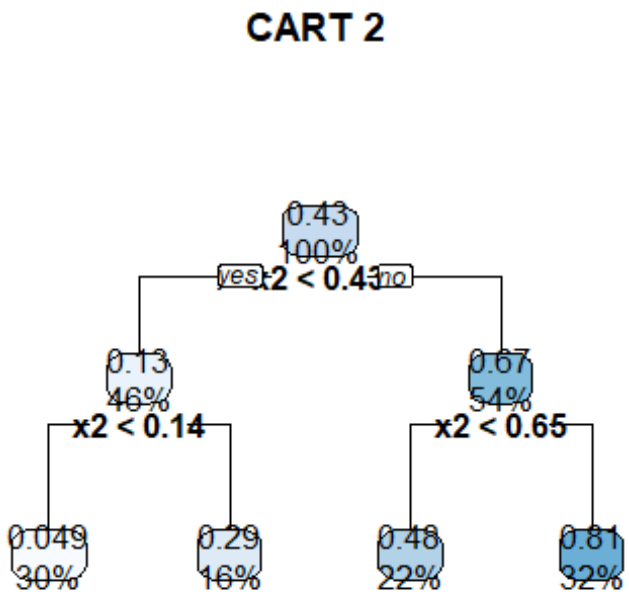



```
x11()
#the full monty in one chart
plot(x2,y, main = "Linear Regression & CART 1")
abline(lm(y~x2))
abline(h=0.1339119)
abline(h=0.6749837)
abline(v=0.4278535)
```

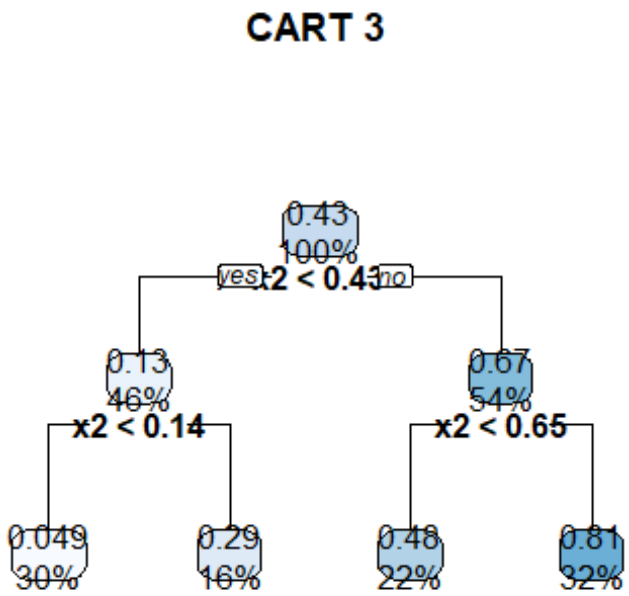



```
#deep dive-----------------------------------------------------

#MSE by no splitting, i.e. forecast mean(y)
sum((y-mean(y))^2)/length(y)#compare this to MSE in cart summary

## [1] 0.09708513

#now use split, use chart & summary data and build function
cart1Forecast = function(x){
  if (x < 0.4278535) (0.1339119)
  else (0.6749837)
}

#cross check first forecast
yx2_pairs = Matrix(c(y,x2),50,2)

m2 = 0
j=0
for (i in (1:50)){
  if(yx2_pairs[i,2]<0.4278535){
    m2 = m2 + yx2_pairs[i,1]
    j=j+1

      }
}
m2=m2/j
m2 #hence, average y is forecasted
```

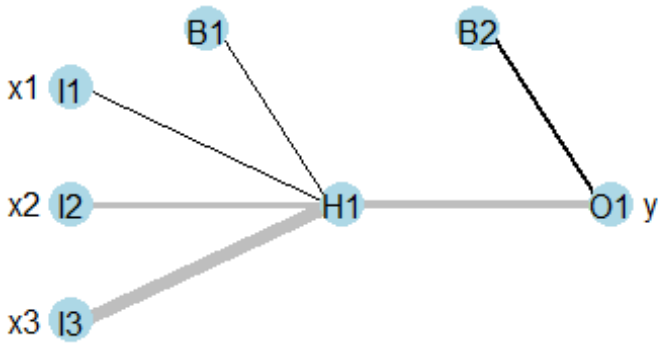
```
## [1] 0.1339119

#test
cart1Forecast(0.8)

## [1] 0.6749837

y_hat_cart1 = apply(as.matrix(x2), MARGIN = 1, FUN = cart1Forecast)
sum((y-y_hat_cart1)^2)/length(y)#compare this to MSE in cart summary

## [1] 0.02436387

23/50*0.01691391 + 27/50*0.03071012

## [1] 0.02436386

#R²
round(cor(lm1$fitted.values,y)^2,4)

## [1] 0.9758

round(cor(y_hat_cart1,y)^2,4)#decrease

## [1] 0.749

# apply your 2nd tree

cart2 = rpart(y~x1+x2+x3, data = df1, maxdepth = 2)
round(cor(lm1$fitted.values,y)^2,4)

## [1] 0.9758

round(cor(rpart.predict(cart2,as.data.frame(cbind(x1,x2,x3)),rules=F),y)^2,4)
#improve

## [1] 0.9525

#but more honest to compare
lm_single = lm(y~x2)
round(cor(lm_single$fitted.values,y)^2,4)

## [1] 0.973

round(cor(rpart.predict(cart2,as.data.frame(cbind(x1,x2,x3)),rules=F),y)^2,4)
#improve

## [1] 0.9525

x11()
#compare this to CART tree result in Larose page 324, Fig. 11.4
rpart.plot(cart2, main = "CART 2")
```



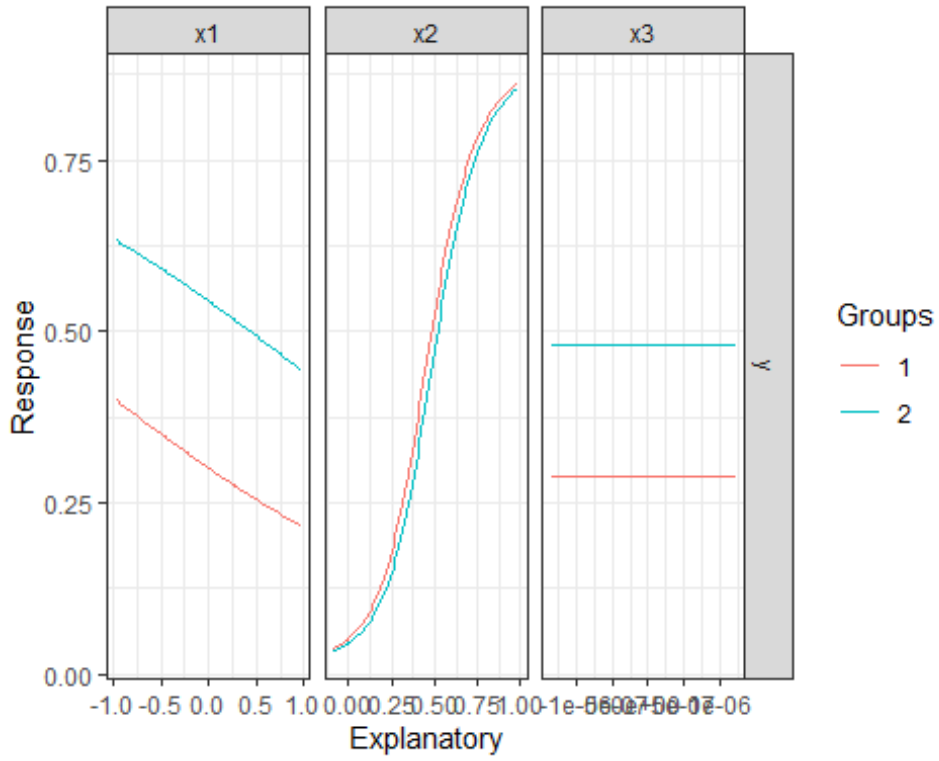

```
# apply your 3rd tree

cart3 = rpart(y~x1+x2+x3, data = df1, maxdepth = 3)
round(cor(lm1$fitted.values,y)^2,4)

## [1] 0.9758

round(cor(rpart.predict(cart3,as.data.frame(cbind(x1,x2,x3)),rules=F),y)^2,4)
#improve

## [1] 0.9525

x11()
#compare this to CART tree result in Larose page 324, Fig. 11.4
rpart.plot(cart3, main = "CART 3")
```



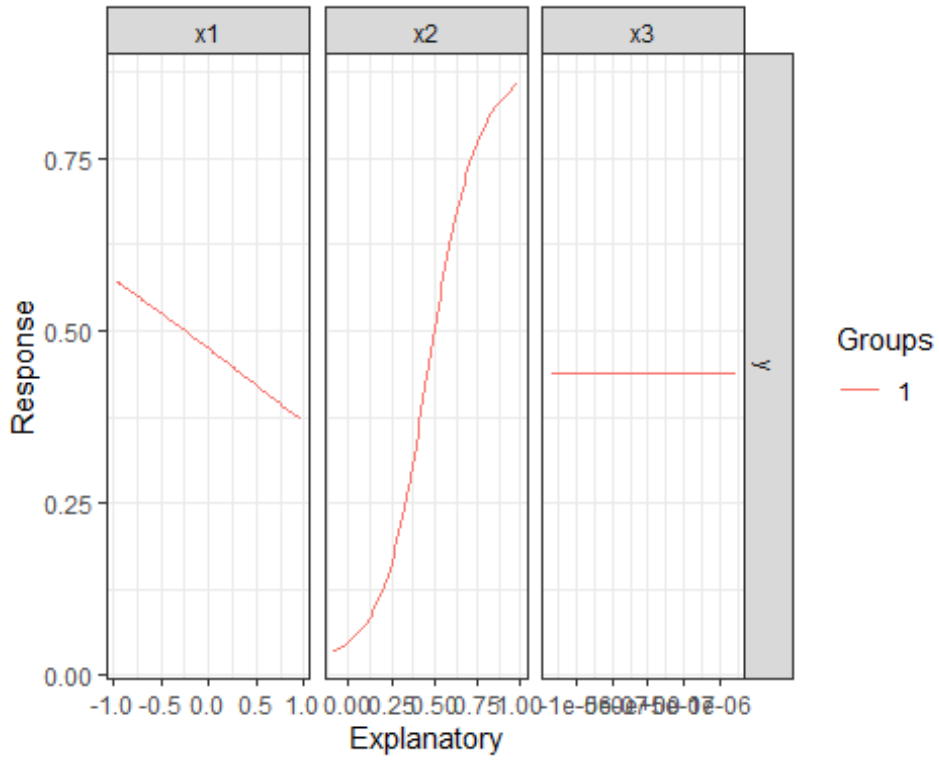

```
# apply your first MLP

set.seed(SEED)
#model as formula
modelform = y~x1+x2+x3

net0 = neuralnet(modelform, data = df1, hidden = 1, err.fct = "sse", lifesign
= "full", act.fct = "logistic",
                 linear.output = F)
#use mostly default values for starters and only one hidden neuron, hence
net0
#can set algorithm = "rprop+", threshold = 0.01) to make default explicit
net0_fc = predict(net0, as.matrix(cbind(x1,x2,x3)), rep = 1, all.units =
FALSE)
#Return output for all units instead of final output only

#Plot the neural network, rough and dirty - old school
x11()
plot(net0)

#nicer is
x11()
plotnet(net0, y_names = "y", circle_cex = 3, node_labs = T, var_labs =
T)#default settings
```




```
#NOTE positive black, grey = ng., this confuses a bit, see that x3 gets kind
of canceled by pos/neg weights

x11()#function y(xi) shown with all other xj fixed at their resp 10/90
percent quantile
lekprofile(net0, group_show = F, group_vals = c(0.4,0.6))
```

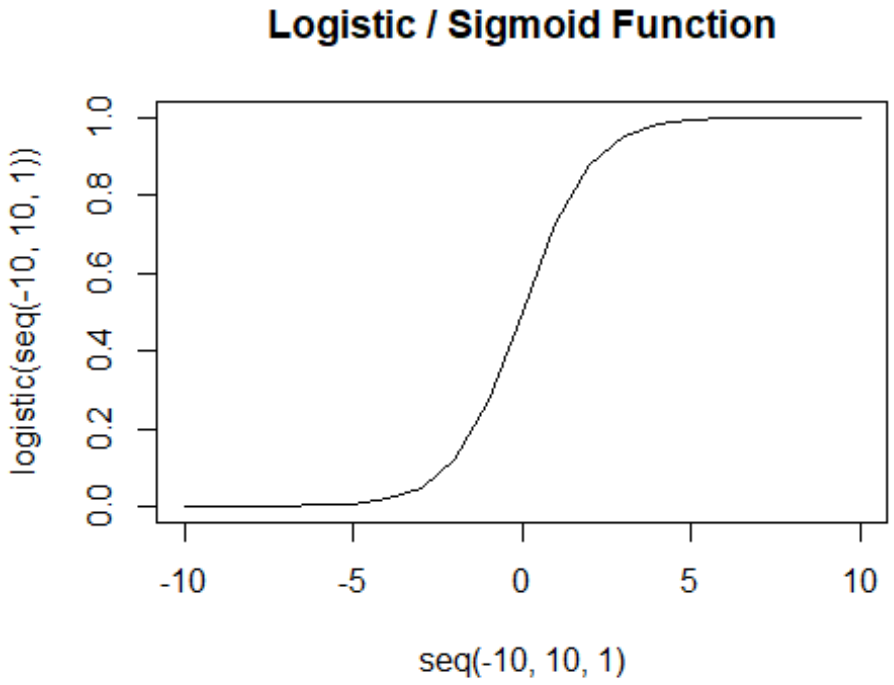



```
#NOTE: Net0 has found out that x1 has a quadratic impact! And that x3 is of
no interest

x11()#same at median values
lekprofile(net0, group_show = F, group_vals = c(0.5))
```




```
#padding on the x-axis to make it readable in full

#compare R² of lm1 and net0
round(cor(lm1$fitted.values,y)^2,4)

## [1] 0.9758

round(cor(net0_fc,y)^2,4)#clear cause net has got more parameters!

##        [,1]
## [1,] 0.9764

# Deep dive into your first MLP with ony one hidden

#understand output from red letter reporting, see that SSE/2 is reported!
net0$result.matrix[1,1]

##      error
## 0.05769369

round(sum((y-net0_fc)^2)/2,5)#note that target fct is 1/2 SSE

## [1] 0.05769

#cause by first derivation get 2*SSE which cancels then against 1/2

#typical activation fct
logistic = function(x){
  1/(1+exp(-x))
}
```

```
x11()
plot(seq(-10,10,1), logistic(seq(-10,10,1)), type = 'l', main = "Logistic /
Sigmoid Function")
```




```
#cross check first forecast
net0_fc[1,1]

## [1] 0.05893039

#we need the follwoing weights
net0$result.matrix[4,1]#Intercept.to.1layhid1

## Intercept.to.1layhid1
##              1.395305

net0$result.matrix[5,1]#x1.to.1layhid1

## x1.to.1layhid1
##      0.2883294

net0$result.matrix[6,1]#x2.to.1layhid1

## x2.to.1layhid1
##      -4.257198

net0$result.matrix[7,1]#x3.to.1layhid1

## x3.to.1layhid1
##      -9.540368
```

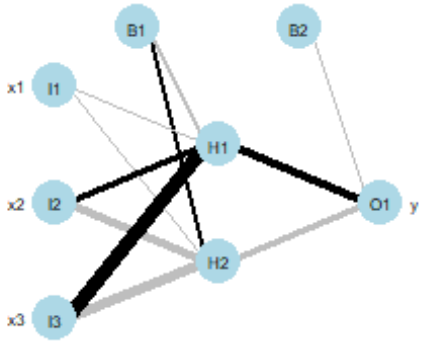
```r
#from input into neuron in first hidden layer
single_hidden_neuron =

logistic(x1[1]*net0$result.matrix[5,1]+x2[1]*net0$result.matrix[6,1]+x3[1]*ne
t0$result.matrix[7,1]
           +net0$result.matrix[4,1])#bX+a
#from first hidden layer to output neuron
logistic(net0$result.matrix[9,1]*single_hidden_neuron+net0$result.matrix[8,1]
)
```

```
## 1layhid1.to.y
##    0.05893039
```

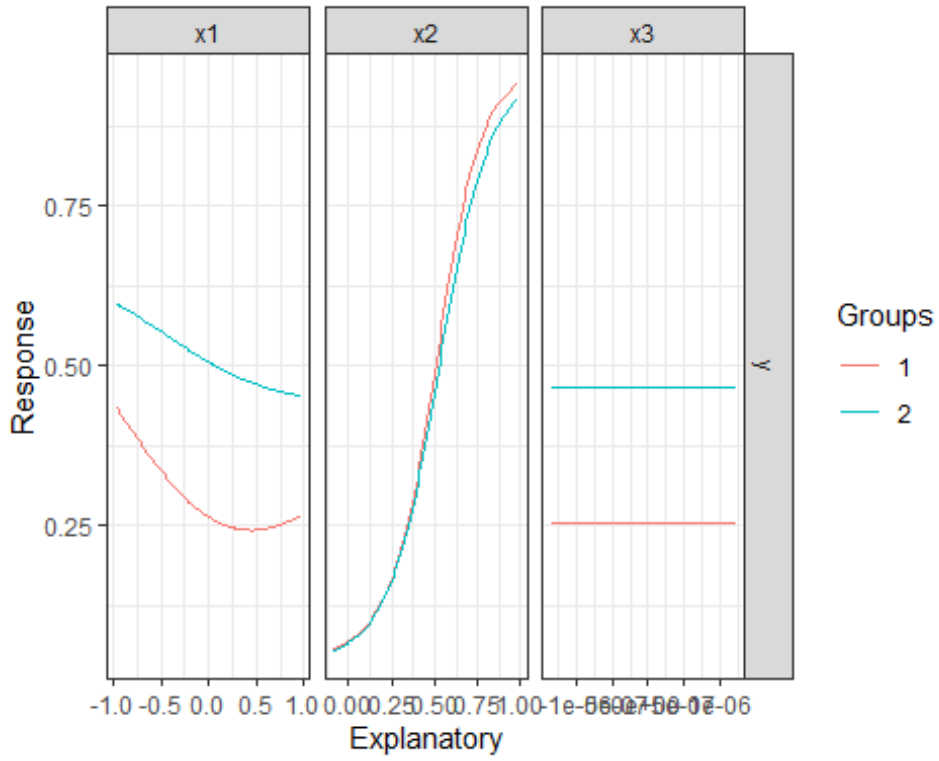
```r
#compare
net0_fc[1,1]
```

```
## [1] 0.05893039
```

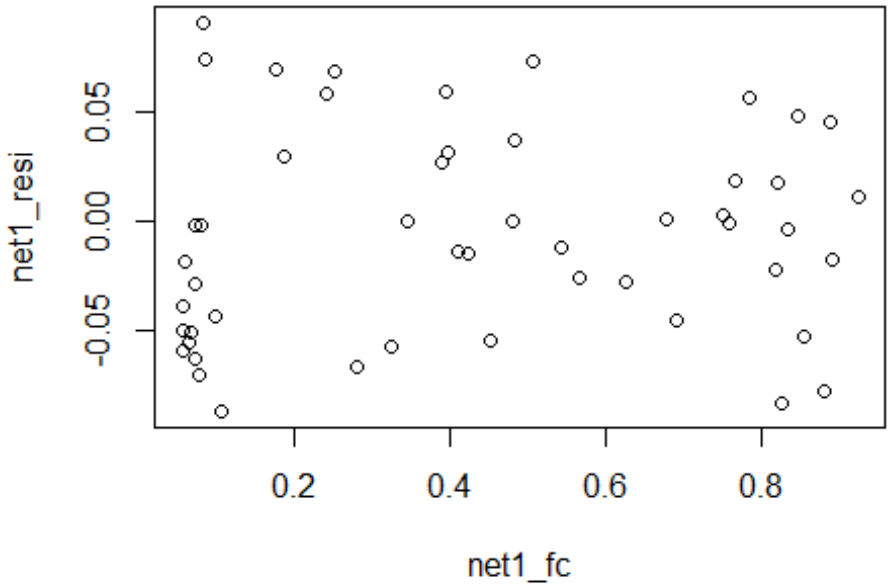
```r
# apply your 2nd MLP with 2 hidden neurons and increase control

#NOW take back control and also change output neuron from linear to signoid,
i.e. MOVE to net1
set.seed(SEED)

#give the net some fctal flexibility to discover that x1 matters squared!!!
#pyramid rule 3 in + 1 out * 2/3 = 8/3 = Ockham! 2 hiddens due to Ockhams
razor
net1 = neuralnet(modelform, data = df1, hidden = 2, stepmax = n_epochs,
                 lifesign = "full", err.fct = "sse", linear.output = F,
                 act.fct = "logistic")
net1_fc = predict(net1, as.matrix(cbind(x1,x2,x3)), rep = 1, all.units =
FALSE)

x11()#controlled setting if a big architecture comes
plotnet(net1, cex_val = 0.5, alpha_val = 1, pad_x = 0.6,#padding on the x-
axis to make it readable in full
        y_names = "y", circle_cex = 3, node_labs = T, var_labs = T)
```



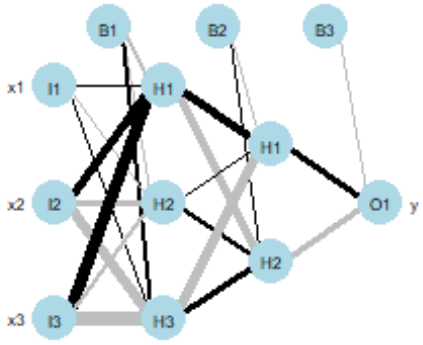

```
#NOTE positive black, grey = neg., this confuses a bit, see that x3 gets kind
of canceled by pos/neg weights

#show the clusters
x11()#function y(xi) shown with all other xj fixed at their resp 10/90
percent quantile
lekprofile(net1, group_show = F, group_vals = c(0.4,0.6))
```



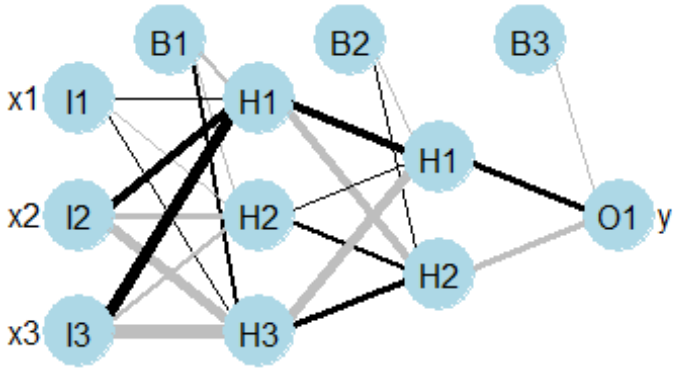

```
#NOTE: Net has found out that x1 has a quadratic impact! And that x3 is of no
value
#compare R² of lm1 and net1
round(cor(lm1$fitted.values,y)^2,4)

## [1] 0.9758

round(cor(net1_fc,y)^2,4)#slight increase

##        [,1]
## [1,] 0.9777

#plot winning net
x11()
plot(net1)

#apply diagnostics
net1_resi = y-net1_fc

##Hetero-----------------------------------------------------
plot(net1_resi, type='l')

x11()
plot(net1_fc,net1_resi)
```



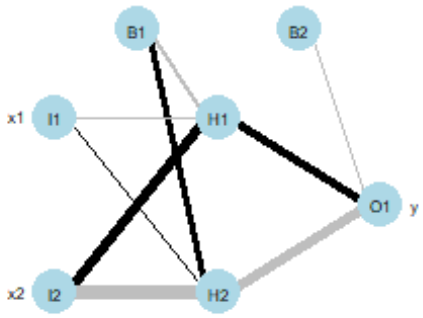

```r
# apply richer MLP

set.seed(SEED)
net2 = neuralnet(modelform, data = df1, hidden = c(3,2), #pyramidal

                 stepmax = n_epochs,# has been shown above as sufficient
                 lifesign = "full",
                 err.fct = "sse", #sse' and 'ce' which stand for the sum of
squared errors and the cross-entropy can be used

                 linear.output = F, #replace linear output neuron by logistic
activation, note the chnage in x2 fct form
                 act.fct = "logistic")#this is the real change to frist try
net2_fc = predict(net2, as.matrix(cbind(x1,x2,x3)), rep = 1, all.units =
FALSE)#Return output for all units instead of final output only

x11()#controlled setting if big net comes
plotnet(net2, cex_val = 0.5, alpha_val = 1, pad_x = 0.6,#padding on the x-
axis to make it readable in full
        y_names = "y", circle_cex = 3, node_labs = T, var_labs = T)
```



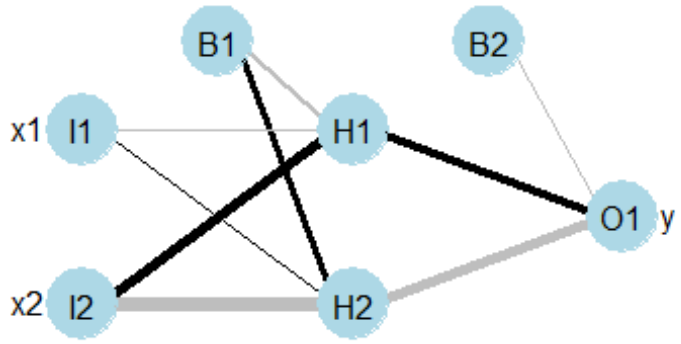

```
#NOTE positive black, grey = ng., this confuses a bit, see that x3 gets kind
of canceled by pos/neg weights
x11()#controlled setting if big net comes
plotnet(net2, y_names = "y")
```



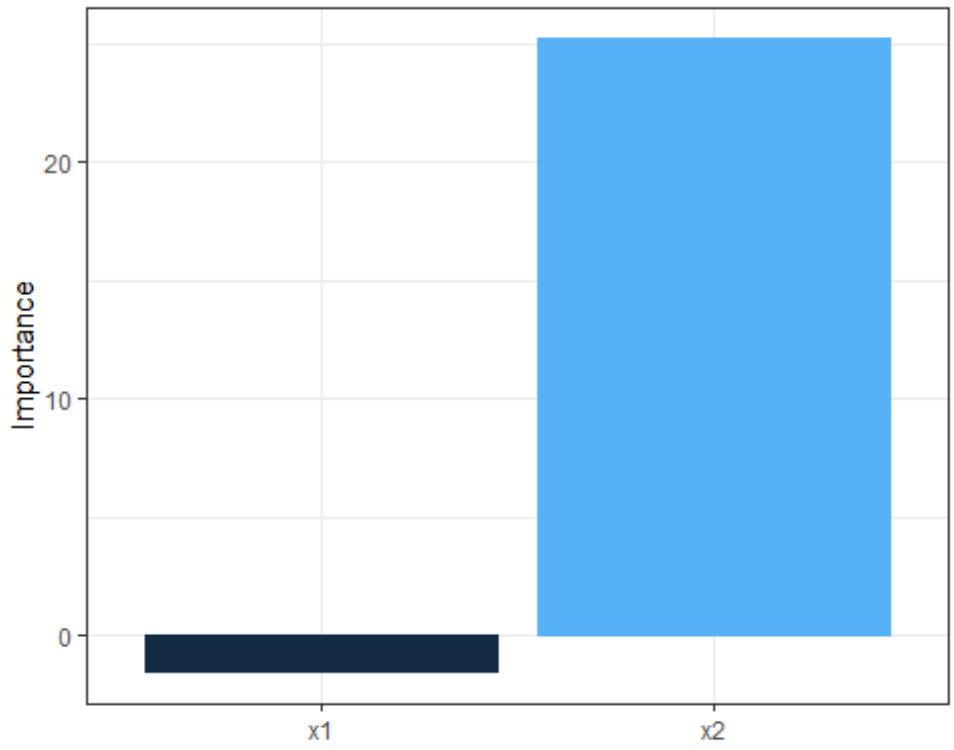
```r
#compare R² of lm1 and net2: NOTE bigger not better
round(cor(lm1$fitted.values,y)^2,4)

## [1] 0.9758

round(cor(net2_fc,y)^2,4)#less than linear model!

##        [,1]
## [1,] 0.9715

#need math to understand this! one hidden layer is enough!!!

# MLP with 2 hidden neurons by using insights from lekprofile that x3 does
not matter

set.seed(SEED)
#model as formula
modelform_selection = y~x1+x2

#give the net some fctal flexibility to discover that x1 matters squared!!!
#pyramid rule 3 in + 1 out * 2/3 = 8/3 = Ockham! 2 hiddens due to Ockhams
razor
net3 = neuralnet(modelform_selection, data = df1, hidden = 2, stepmax =
n_epochs,
                 lifesign = "full", err.fct = "sse", linear.output = F,
                 act.fct = "logistic")
net3_fc = predict(net3, as.matrix(cbind(x1,x2)), rep = 1, all.units = FALSE)

x11()#controlled setting if a big architecture comes
plotnet(net3, cex_val = 0.5, alpha_val = 1, pad_x = 0.6,#padding on the x-
axis to make it readable in full
        y_names = "y", circle_cex = 3, node_labs = T, var_labs = T)
```



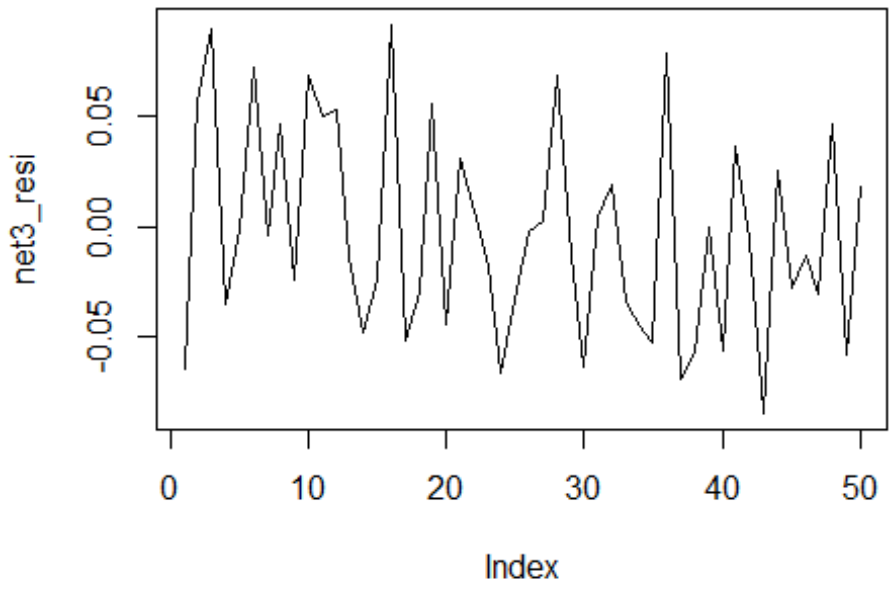

```
#NOTE positive black, grey = neg., this confuses a bit, see that x3 gets kind
of canceled by pos/neg weights

#compare R² of lm1 and net3
round(cor(lm1$fitted.values,y)^2,4)

## [1] 0.9758

round(cor(net3_fc,y)^2,4)#slight decrease

##        [,1]
## [1,] 0.9775

#plot winning net
x11()
plot(net3)

#nicer is
x11()
plotnet(net3, y_names = "y")#default settings
```




```
#NOTE positive black, grey = ng., this confuses a bit, see that x3 gets kin

x11()
olden(net3)
```



```r
#apply diagnostics
net3_resi = y-net3_fc

##Hetero-----------------------------------------------------
plot(net3_resi, type='l')
```




```r
#end
```